

05 - Depth First Traversal

A differenza del problema del broadcast in cui si deve condividere un'informazione con tutti i nodi della rete, si consideri la situazione in cui c'è una risorsa da distribuire in modo *mutualmente esclusivo* tra i nodi. In particolare esiste una sola copia della risorsa e in un qualunque istante temporale esiste un solo nodo che la possiede.

In particolare si studia una strategia di condivisione nota come **depth first traversal**. L'idea alla base prevede che ogni nodo cerca di inoltrare il token ai suoi vicini fino a quando tutti i nodi a lui adiacenti hanno già ricevuto il token. Quando un nodo non può inoltrare il token in "avanti", lo rimanda "indietro" al nodo che glielo ha inoltrato precedentemente.

Il problema

Formalmente, il problema è definito dalla seguente tripla $P = \langle P_{init}, P_{final}, R \rangle$ dove, sia $\forall u \in V$, $received(u)$ un registro che contiene True se u ha ricevuto il token, False altrimenti.

- P_{init} : $\exists! x \in V : received(x) = \text{True} \wedge \forall y \neq x : received(y) = \text{False}$, ossia, solo l'initiator ha il token;
- P_{final} : $\forall x \in V : received(x) = \text{True}$, tutti i nodi hanno ricevuto il token seguendo l'ordine di una DFS.
- $R_{dfs} = \begin{cases} \text{Total Reliability (TR)} \\ \text{Bidirectional Link (BL)} \\ \text{Connectivity (CN)} \\ \text{Unique Initiator (UI)} \end{cases}$

Protocollo BACK

Il protocollo più semplice che risolve il problema DFT è il seguente:

1. Quando un nodo riceve il token per la *prima volta*, ricorda il nodo che gli ha inviato il token (il padre) e inoltra il token ad uno dei suoi vicini non visitati con il messaggio **TOKEN**, aspettando la sua risposta;
2. Quando il vicino riceve il token, se lo ha già ricevuto lo ritorna immediatamente con un messaggio **BACK-EDGE**, altrimenti lo inoltra a tutti i suoi vicini in modo sequenziale, per poi ritornarlo al nodo da cui lo ha ricevuto con un messaggio **RETURN**;
3. Quando il nodo riceve il messaggio **RETURN** dal suo vicino, inoltra il token ad un altro vicino;
4. Infine, quando il nodo finisce tutti i vicini, invia il messaggio **RETURN** al nodo da cui inizialmente aveva ricevuto il token, ossia il nodo che ha segnato come padre.

Si formalizza la seguente strategia. Il protocollo fa uso degli stati seguenti

- $S = \{\text{initiator}, \text{idle}, \text{visited}, \text{done}\}$

- $S_{init} = \{\text{initiator}, \text{idle}\}$
- $S_{term} = \{\text{done}\}$

Il comportamento dei nodi è descritto dalle seguenti regole

```

if state == "INIT":
    spontaneously:
        unvisited = N(x)
        initiator = True
        VISIT

if state == "IDLE":
    receiving("T"):
        entry = sender
        unvisited = N(x) - {sender}
        initiator = False
        VISIT

if state == "VISITED":
    receiving("T"):
        unvisited = unvisited - {sender}
        send("Backedge") to {sender}
    receiving("Return"):
        VISIT
    receiving("Backedge"):
        VISIT

if state == "DONE":
    None

```

dove VISIT è la procedura utilizzata dai nodi per implementare la DFT ed è la seguente

```

Procedure VISIT
    if unvisited != {}:
        next = extract(unvisited)
        send("T") to next
        state = "VISITED"
    else:
        if not(state == "INITIATOR"):
            send("Return") to entry
        state = "DONE"

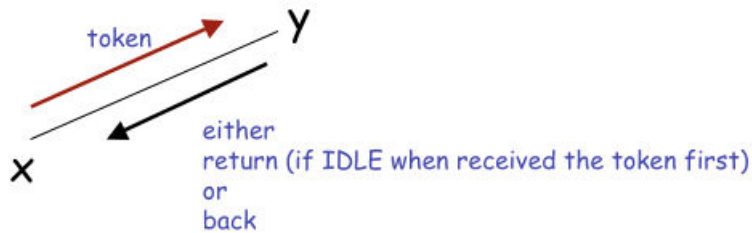
```

Message complexity

Si osserva che nel protocollo appena descritto esistono tre tipologie di messaggi, che sono:

- Messaggi TOKEN: usati per scambiare il token;
- Messaggi BACK-EDGE: usati per indicare che un nodo ha già ricevuto il token;

- **Messaggi RETURN:** usati per restituire il token dopo averlo ricevuto per la prima volta. Dato un arco $(x, y) \in E$, dove x invia il token ad y , sono possibili le seguenti situazioni



allora su ogni arco passano esattamente due messaggi. Dunque la message complexity vale

$$M(\text{BACK}) = 2m$$

Lower bound alla message complexity

Si osserva che non si può progettare un protocollo per il problema DFT che utilizza $o(m)$ messaggi. Infatti risolvendo DFT si risolve anche il problema broadcast, dunque il lower bound del broadcast si riflette anche su questo problema:

$$M(\text{DFT}/R_{dft}) \geq m$$

E quindi, in termini di message complexity il protocollo BACK risulta essere asintoticamente ottimo e, in particolar modo, $M(\text{DFT}/R_{dft}) = \Theta(m)$.

Ideal Time Complexity

Come sempre, si ipotizza di trovarsi in un sistema sincrono, con i vincoli di synchronized clock e unitary bounded delay.

Si osserva che nel protocollo BACK ogni nodo esplora in modo sequenziale tutti i suoi vicini. Questo significa la ideal time complexity è pari a

$$T(\text{BACK}) = M(\text{DFT}/R_{dft}) = \Theta(m)$$

Notiamo però che in termini di ideal time complexity il problema DFT ha un lower bound di $\Omega(n)$, questo perché ogni nodo deve venir visitato sequenzialmente, iniziando dal nodo initiator. Quindi esiste un gap abbastanza significativo tra il lower bound e la ideal time complexity del protocollo discusso.

 Note

Quando si misura l'ideal time, si considerano solamente esecuzioni sincrone, ma quando si misura la message complexity e la correttezza di un protocollo, bisogna considerare *ogni* possibile schedule degli eventi, specialmente le esecuzioni asincrone.

Protocollo BACK+

Utilizzando il protocollo BACK in grafi *densi*, ovvero grafi con molti archi, la maggior parte del tempo viene speso dai nodi aspettando la ricezione dei messaggi BACK-EDGE. Per evitare questo spreco di tempo, si progetta un protocollo che non utilizza i messaggi BACK-EDGE. In particolare, si effettuano le seguenti modifiche al protocollo BACK:

1. Quando un nodo x riceve il token, informa tutto il suo vicinato $N(x)$ che possiede il token, ossia che x è stato visitato;
2. Tutti i nodi in $N(x)$ inviano un messaggio di ACK al nodo x ; I nodi in $N(x)$ sapranno quindi di non dover inoltrare nuovamente il token ad x .
3. Dopo aver ricevuto tutti gli ACKs, il nodo x invia il token ad uno dei suoi vicini che non sono ancora visitati.

Tale protocollo prende il nome di BACK+. La differenza sostanziale rispetto al protocollo BACK è che i "back-edges" sono scoperti in parallelo e, in particolar modo, che un'entità pronta ad inoltrare il token ai suoi vicini sa quali tra essi sono stati già visitati.

Message complexity

Si osserva che tutti i nodi tranne l'initiator ricevono un solo messaggio TOKEN e inviano un solo messaggio RETURN. L'initiator non può ricevere nessun messaggio TOKEN e non invia nessun messaggio RETURN. Dunque si hanno esattamente $2(n-1)$ messaggi inviati di tipo RETURN e TOKEN.

Ogni nodo invia un messaggio VISITED a tutti i nodi a lui adiacenti, tranne al nodo da cui ha ricevuto il messaggio. Allora il numero totale di messaggi VISITED è pari a

$$|N(s)| + \sum_{x \in V \setminus \{s\}} |N(x)| - 1 = 2m - (n - 1)$$

Infine, si osserva che ogni messaggio VISITED implica un corrispondente messaggio di ACK. Dunque il numero totale di messaggi è pari a

$$M(\text{BACK+}) = \underbrace{2(2m - (n - 1))}_{\text{VISITED e ACK}} + \underbrace{2(n - 1)}_{\text{TOKEN e RETURN}} = 4m$$

Ideal Time Complexity

Si assuma di eseguire il protocollo BACK+ in un sistema sincrono con un delay di trasmissione pari a $\Delta = 1$. Allora vale che:

- I messaggi TOKEN e RETURN sono inviati in modo sequenziale, impiegando tempo $2(n - 1)$;
- I messaggi ACK e VISITED sono inviati in modo parallelo. In particolare, ogni nodo impiega 2 unità di tempo per queste operazioni. Il tempo totale impiegato tra l'invio e la ricezione di questi messaggi è quindi pari a $2n$.

In totale si ottiene

$$T(\text{BACK}+) = 2(n - 1) + 2n = 4n - 2$$

il che risulta essere asintoticamente ottimale.

Ulteriori modifiche

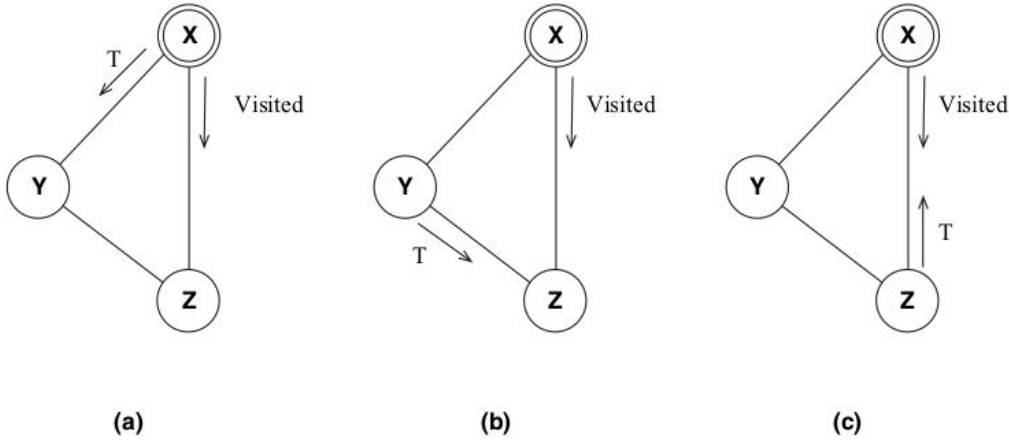
Ci si chiede se si può ulteriormente ridurre il numero di messaggi trasmessi durante l'esecuzione del protocollo. In particolar modo, si osserva che non si può fare a meno di inviare i messaggi contenenti il token ed il messaggio di return affinché un generico protocollo porti a termine il task distribuito, e che i messaggi VISITED fanno sì che un agente possa eventualmente diminuire il numero di vicini a cui inoltrare il token. Il target di quest'analisi saranno quindi i messaggi di acknowledgment.

Ci si chiede quindi cosa succederebbe se non venissero mandati i messaggi di ACK. Si consideri un'entità x , che notifica i nodi nel suo vicinato la ricezione del token mediante un messaggio VISITED. Se non si usassero più i messaggi di ACK, x procederebbe immediatamente inoltrando il token ad uno dei suoi vicini. Si assuma che, dopo una quantità di tempo finito, il token raggiunga, per la prima volta, un vicino z di x . E' possibile che il messaggio VISITED inviato da x verso z non sia ancora giunto a destinazione, per via dei ritardi di comunicazione.

In tal caso, z non sarebbe a conoscenza del fatto che x è stato già visitato, ed invierebbe quindi il messaggio contenente il token ad esso. Ossia, si manderebbe nuovamente un messaggio contenente il token su un back-edge, rimuovendo il miglioramento ottenuto con il protocollo BACK+ rispetto al protocollo originale BACK.

L'algoritmo appena descritto però risulta differente dal protocollo originale, essendo che si usano i messaggi VISITED invece dei messaggi BACKEDGE, e lo scenario appena

descritto può non verificarsi ad ogni interazione tra agenti.



Ma, se tale situazione si verifica, z riceverà prima o poi il messaggio di VISITED da x per via della total reliability. z può quindi capire il suo "errore", ossia l'invio del token ad un nodo già visitato, fingendo che nulla sia accaduto (eccetto per lo spreco di un messaggio contenente il token), e continuare come se tale messaggio non fosse mai stato inviato. Dall'altro lato, x , dopo aver ricevuto il token da z , capirà a sua volta che z ha effettuato un errore, ignorando quindi tale messaggio. Inoltre, da questa interazione, x può venir a sapere, sempre se non lo ha fatto in precedenza, che z è stato già visitato, e può quindi rimuoverlo dalla sua lista di vicini non visitati.

Nonostante queste modifiche, la correttezza del protocollo rimane invariata, ma gli errori appena descritti influiscono sul numero dei messaggi inviati. Sia BACK++ il nome del protocollo modificato. Si effettua ora un'analisi dei suoi costi.

Come nel protocollo precedente, i messaggi "corretti" contenenti il token e i messaggi di RETURN hanno un peso sulla message complexity di $2(n - 1)$, e i messaggi VISITED, come prima, sono $2m - n + 1$ in totale.

Si considerino ora i messaggi inviati "erroneamente": ogni "errore" costa un messaggio. Il numero di errori può essere molto grande. Infatti, ritardi di trasmissione sfavorevoli possono far sì che questi si verificano su ogni back-edge. Su ciascun back edge, possono verificarsi al più due errori, uno in ciascuna direzione. In altre parole, possono esservi al più $2(m - (n - 1))$ messaggi che trasmettono inutilmente (e quindi "incorrettamente") il token. Sommando il tutto si ottiene che

$$M(\text{Back++}) \leq 4m - n + 1$$

Si consideri ora il tempo. Rispetto al caso precedente, si ha un miglioramento dovuto al fatto che i messaggi di ACK non vengono più inviati, risparmiando n unità di tempo.

Essendo inoltre che non vi sono più messaggi di ACK da attendere dopo la trasmissione dei messaggi VISITED ai nodi adiacenti, un'entità può inviare il token ad un suo vicino nello stesso istante in cui trasmette tali messaggi VISITED ai nodi ad esso adiacenti. Se tutti i vicini di tale entità risultano essere già visitati, allora l'entità invierà il messaggio di RETURN al padre nello stesso istante in cui trasmette i messaggi VISITED.

In sintesi, l'invio dei VISITED avviene simultaneamente all'invio di un RETURN o del token, risparmiando ulteriori n unità di tempo.

Complessivamente, senza considerare gli errori, il tempo totale è $2(n - 1)$.

Ora, si vorrebbe studiare l'ideal time complexity considerando anche i messaggi erroneamente inviati, ma si osserva che, sotto le ipotesi di synchronized clocks e unitary bounded delay, necessarie al fine di stimare tale misura, tali errori in una data esecuzione non possono mai verificarsi. Questo perché gli errori si verificano nel caso di ritardi di comunicazione arbitrariamente lunghi, i quali non si verificherebbero mai essendo che la propagazione di ciascun messaggio su un canale di comunicazione richiede una singola unità di tempo. Quindi

$$T(\text{Back} + +) = 2n - 2$$